

Instructor – Zaheredin S.

Chapter Five Contents!



- 5.1 Shell sort
- 5.2 Heap sort

Chapter Five Contents!



- 5.3 Quick sort
- 5.4 Merge sort

5.1 - Shell Sort

What is "Shell Sort"?

- Shell sort is a highly efficient sorting algorithm.
- It is the generalization of insertion sort, which overcomes the drawbacks of insertion sort by comparing elements separated by a gap of several positions.
- It has improved the average time complexity of insertion sort.
- In insertion sort, at a time, elements can be moved ahead by one position only. To move an element to a far-away position, many movements are required that increase the algorithm's execution time.

Cont...

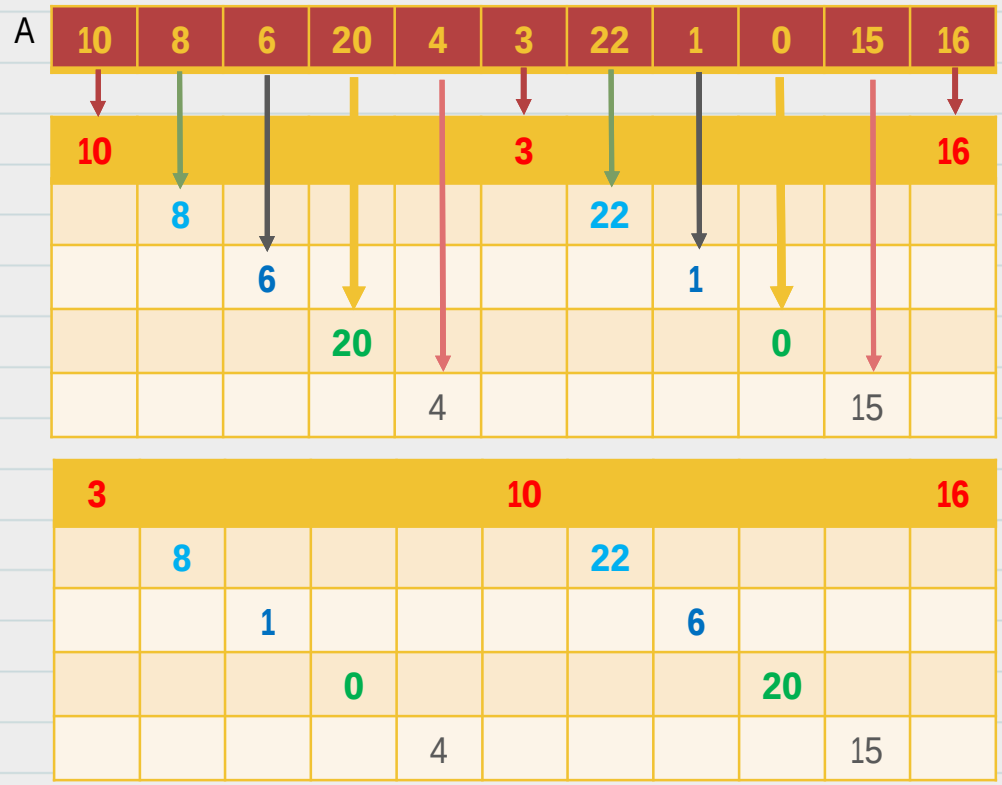
- But shell sort overcomes this drawback of insertion sort. It allows the movement and swapping of far-away elements as well.
- The original array is first divided into a number of smaller subarrays and sorts the elements in subarrays that are far away from each other, then it subsequently reduces the gap between them. This gap is called as interval.
- Once the algorithm reaches the point where the gap value is 1, it uses insertion sort.

Cont...

- These subarrays are sorted, and then they are combined into the overall array and this is sorted.

Let us see an example...

Cont...



N=11

gap1=floor(N/2)

gap1=floor(11/2)=5

Before sorting with gap1=5

Swap 10 & 3

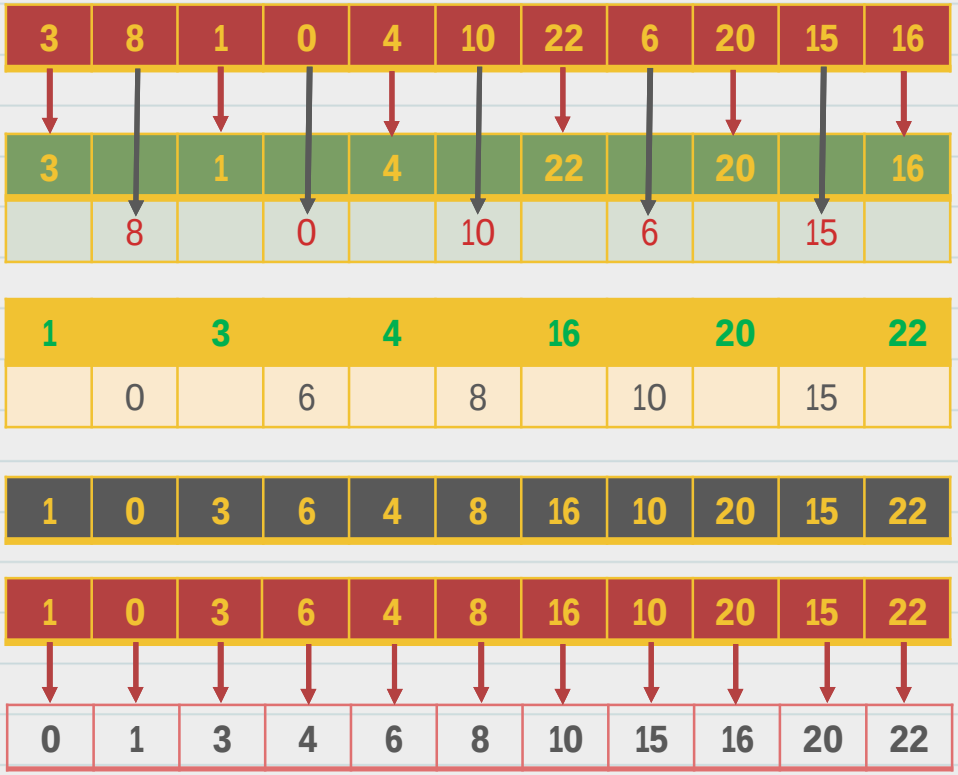
No swap

Swap 6 & 1

Swap 20 & 0

No swap

Cont...


3 8 1 0 4 10 22 6 20 15 16
Array after gap 5
3 1 4 22 20 16
gap2=floor(gap1/2)=5/2=2
8 0 10 6 15
Before sorting with gap2
1 3 4 16 20 22
0 6 8 10 15
Swap with correct order
1 0 3 6 4 8 16 10 20 15 22
Array after gap2=2
1 0 3 6 4 8 16 10 20 15 22
gap3=floor(2/2)=1
For gap3=1 Use insertion sort
0 1 3 4 6 8 10 15 16 20 22
Sorted array after gap3=1

Cont...

Time Complexity of Shell Sort	
Worst Case	$O(n^2)$
Average Case	$O(n^{5/4})$
Best Case	$O(n^{3/2})$

QUIZE (5%)

Q1: Sort the following elements using shell sort.

33	31	40	8	12	17	25	42

5.2 - Heap Sort

Before discussing about the heap sort, let's first see a brief description of Heap.

What is a "heap"?

- A **heap** is a complete binary tree, and the binary tree is a tree in which the node can have the utmost two children.
- A complete binary tree is a binary tree in which all the levels except the last level(leaf-node), should be completely filled, and all the nodes should be left-justified.

Cont...

What is "Heap Sort"?

- **Heap sort** is one of the sorting algorithms used to arrange a list of elements in order and it use tree concepts.
- It is a comparison-based sorting technique based on Binary Heap data structure.
- It is similar to selection sort.
- It processes the elements by creating the min-heap or max-heap using the elements of the given array.

Cont...

- Min-heap or max-heap represents the ordering of array in which the root element represents the minimum or maximum element of the array.
- The concept of heap sort is to eliminate the elements one by one from the heap part of the list, and then insert them into the sorted part of the list.

In heap sort, basically, there are two steps involved in the sorting of elements.

1. Creation of a heap by adjusting the elements of the array.

Cont...

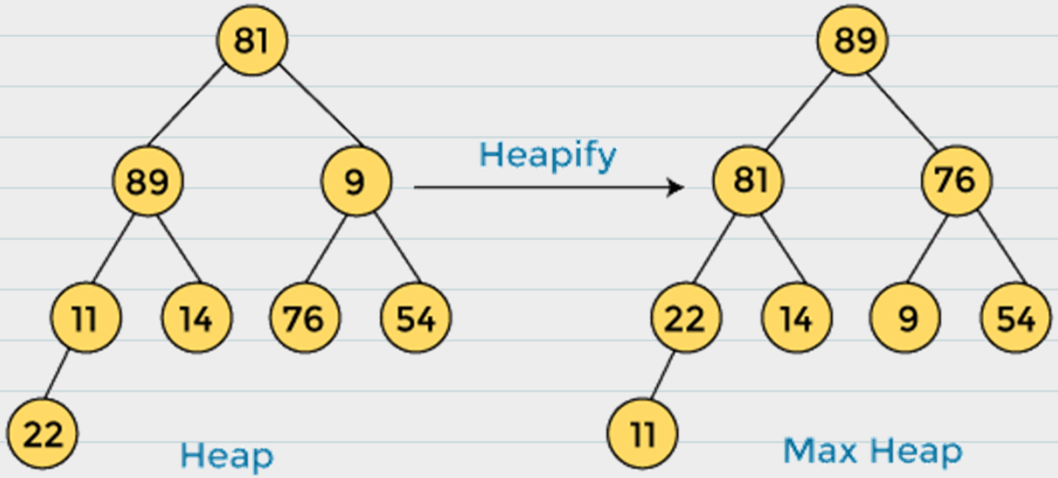
2. After the creation of heap, now remove the root element of the heap repeatedly by shifting it to the end of the array, and then store the heap structure with the remaining elements.

Example: let's take an unsorted array and try to sort it using heap sort.

81	89	9	11	14	76	54	22

- First, we have to construct a heap from the given array and convert it into max heap.

Cont...

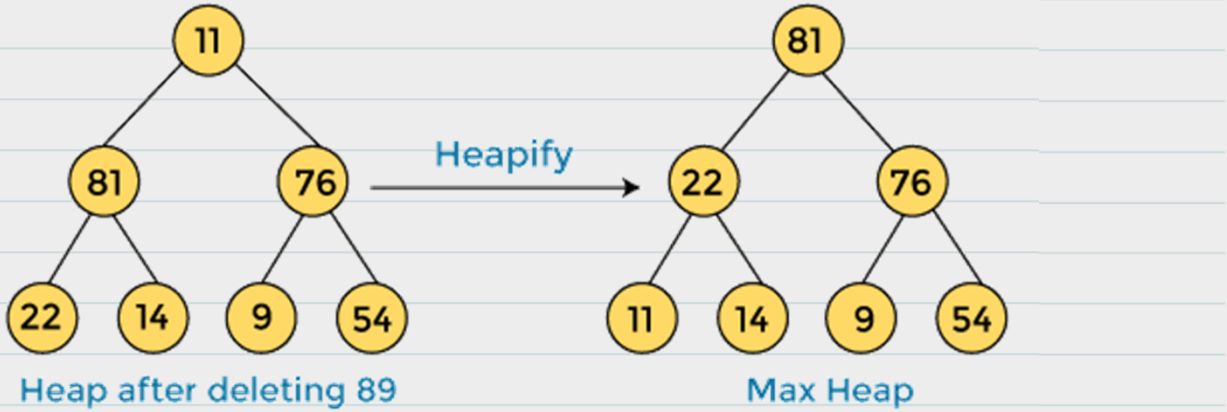


- After converting the given heap into max heap, the array elements are -

89	81	76	22	14	9	54	11

Cont...

- Next, we have to delete the root element (89) from the max heap. To delete this node, we have to swap it with the last node, i.e. (11). After deleting the root element, we again have to heapify it to convert it into max heap.



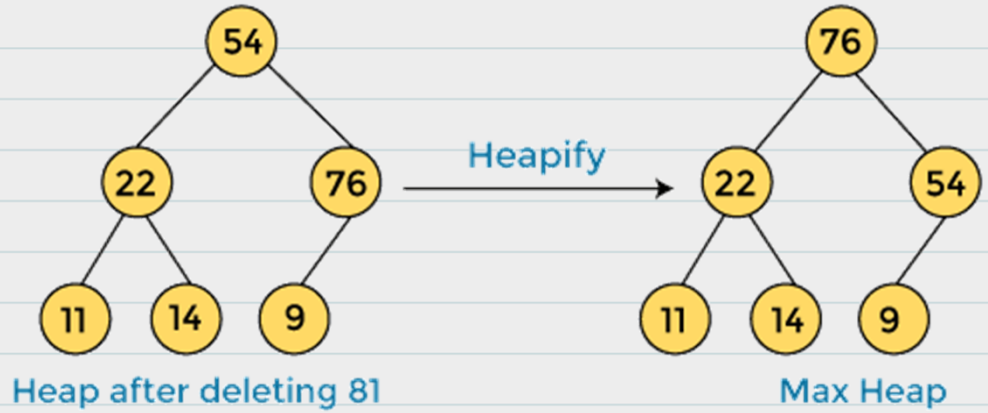
Cont...

- After swapping the array element 89 with 11, and converting the heap into max-heap, the elements of array are –



- In the next step, again, we have to delete the root element (81) from the max heap. To delete this node, we have to swap it with the last node, i.e. (54). After deleting the root element, we again have to heapify it to convert it into max heap.

Cont...

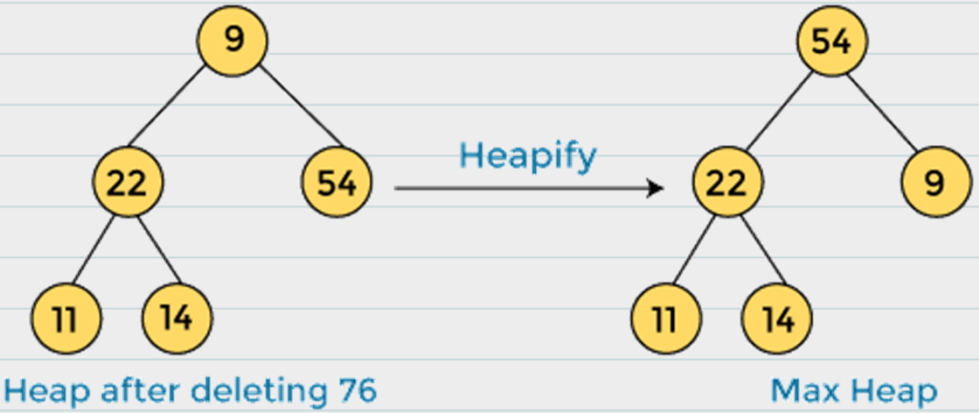


- After swapping the array element 81 with 54 and converting the heap into max-heap, the elements of array are -

76	22	54	11	14	9	81	89

Cont...

- In the next step, we have to delete the root element (76) from the max heap again. To delete this node, we have to swap it with the last node, i.e. (9). After deleting the root element, we again have to heapify it to convert it into max heap.



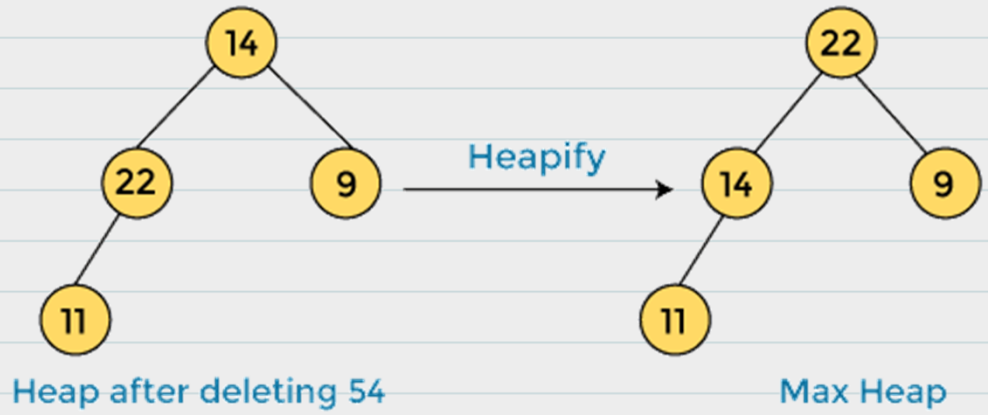
Cont...

- After swapping the array element 76 with 9 and converting the heap into max-heap, the elements of array are –



- In the next step, again we have to delete the root element (54) from the max heap. To delete this node, we have to swap it with the last node, i.e. (14). After deleting the root element, we again have to heapify it to convert it into max heap.

Cont...



- After swapping the array element 54 with 14 and converting the heap into max-heap, the elements of array are -

22	14	9	11	54	76	81	89

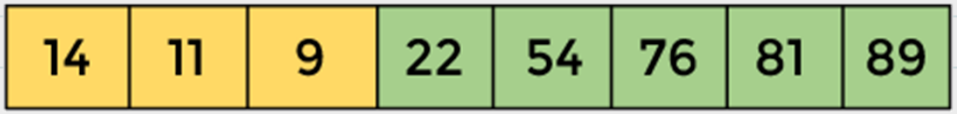
Cont...

- In the next step, again we have to delete the root element (22) from the max heap. To delete this node, we have to swap it with the last node, i.e. (11). After deleting the root element, we again have to heapify it to convert it into max heap.



Cont...

- After swapping the array element 22 with 11 and converting the heap into max-heap, the elements of array are –



- In the next step, again we have to delete the root element (14) from the max heap. To delete this node, we have to swap it with the last node, i.e. (9). After deleting the root element, we again have to heapify it to convert it into max heap.

Cont...

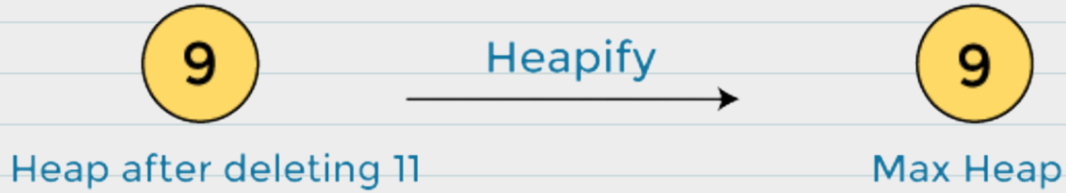


- After swapping the array element 14 with 9 and converting the heap into max-heap, the elements of array are –

11	9	14	22	54	76	81	89

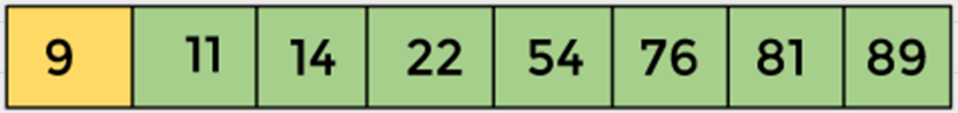
Cont...

- In the next step, again we have to delete the root element (11) from the max heap. To delete this node, we have to swap it with the last node, i.e. (9). After deleting the root element, we again have to heapify it to convert it into max heap.

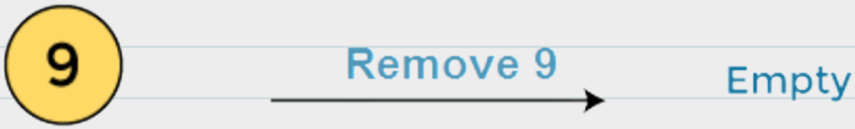


- After swapping the array element 11 with 9, the elements of array are -

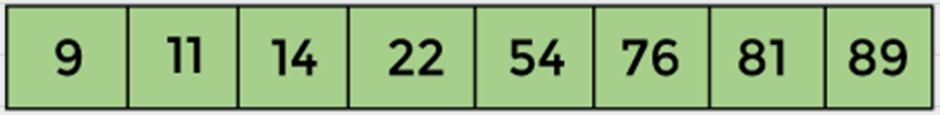
Cont...



- Now, heap has only one element left. After deleting it, heap will be empty.



- After completion of sorting, the array elements are –



- Now, the array is completely sorted.

Cont...

Time Complexity of Heap Sort	
Worst Case	O(n log n)
Average Case	O(n log n)
Best Case	O(n)

Assignment 4

Q2: Sort the following elements using heap sort. NB: heapify using min or max heap

10	9	7	4	2	3

5.3 - Quick sort

- It is a faster and highly efficient sorting algorithm.
- It follows the divide and conquer approach. Divide and conquer is a technique of breaking down the algorithms into sub problems, then solving the sub problems, and combining the results.

❖ Divide

- In Divide, first pick a pivot element. After that, partition or rearrange the array into two sub-arrays such that each element in the left sub-array is less than or equal to the pivot element and each element in the right sub-array is larger than the pivot element.

Cont...

❖ **Conquer**

- Recursively, sort two subarrays with Quicksort.

❖ **Combine**

- Combine the already sorted array.

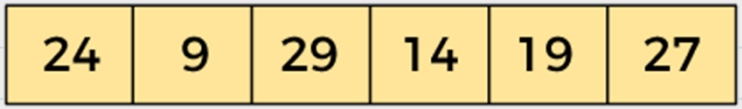
Choosing the pivot

Some of the ways of choosing a pivot are as follows

1. Pivot can be random
2. Pivot can either be the rightmost element or the leftmost element
3. Select median as the pivot element

Cont...

Example: let's take an unsorted array.

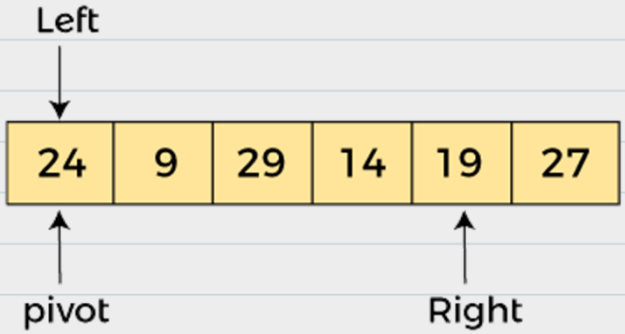


- In the given array, we consider the leftmost element as pivot. So, in this case, a[left] = 24, a[right] = 27 and a[pivot] = 24.
- Since, pivot is at left, so algorithm starts from right and move towards left.

Cont...

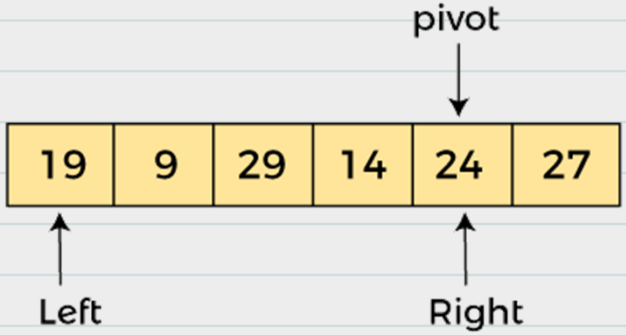


- Now, a[pivot] < a[right], so algorithm moves forward one position towards left, i.e.



Cont...

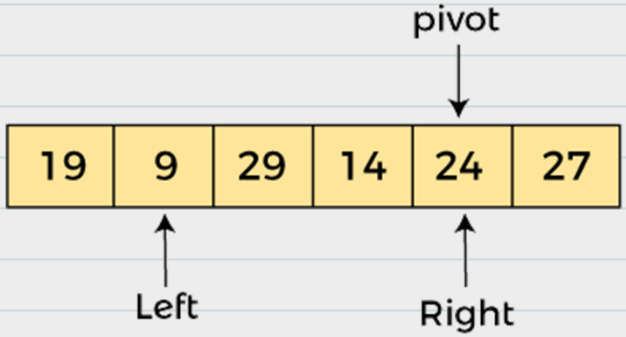
- Now, a[left] = 24, a[right] = 19, and a[pivot] = 24.
- Because, a[pivot] > a[right], so, algorithm will swap a[pivot] with a[right], and pivot moves to right, as –



- Now, a[left] = 19, a[right] = 24, and a[pivot] = 24. Since, pivot is at right, so algorithm starts from left and moves to right.

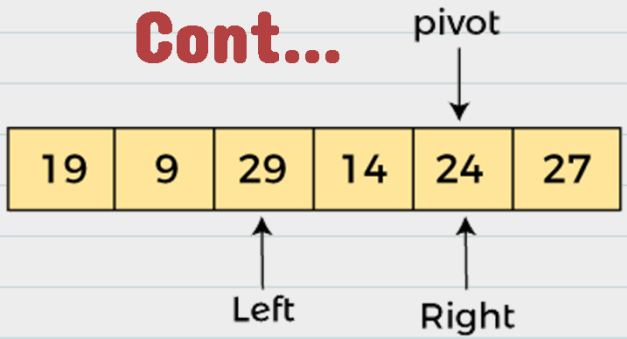
Cont...

- As a[pivot] > a[left], so algorithm moves one position to right as –

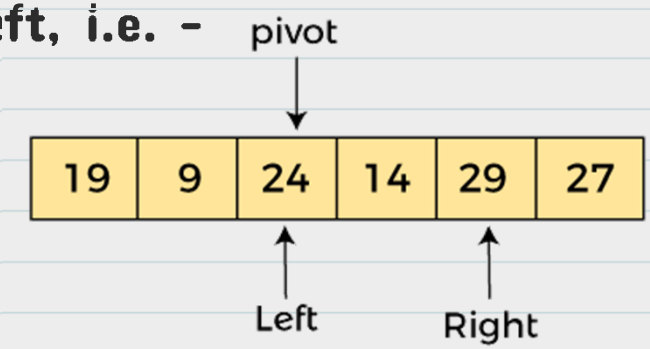


- Now, a[left] = 9, a[right] = 24, and a[pivot] = 24. As a[pivot] > a[left], so algorithm moves one position to right as -

Cont...

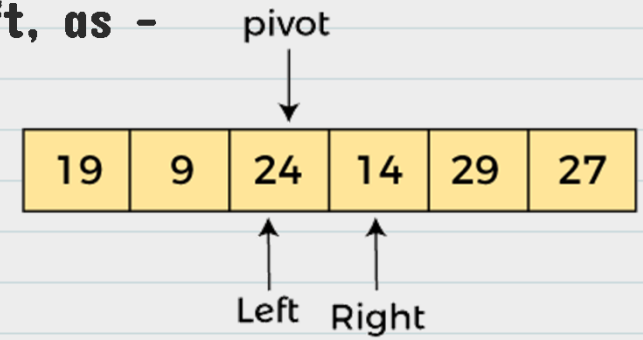


- Now, a[left] = 29, a[right] = 24, and a[pivot] = 24. As a[pivot] < a[left], so, swap a[pivot] and a[left], now pivot is at left, i.e. -



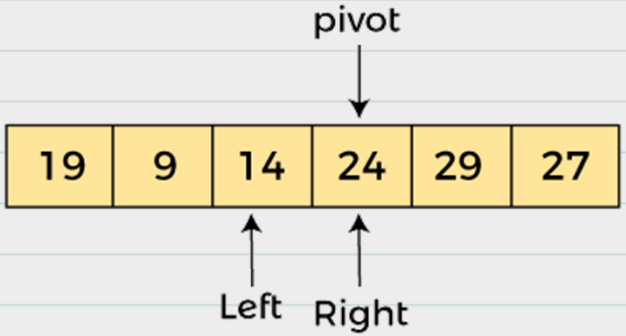
Cont...

- Since, pivot is at left, so algorithm starts from right, and move to left. Now, a[left] = 24, a[right] = 29, and a[pivot] = 24. As a[pivot] < a[right], so algorithm moves one position to left, as -

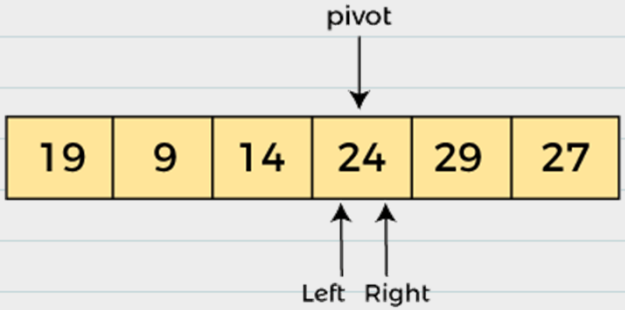


- Now, a[pivot] = 24, a[left] = 24, and a[right] = 14. As a[pivot] > a[right], so, swap a[pivot] and a[right], now pivot is at right, i.e. -

Cont...

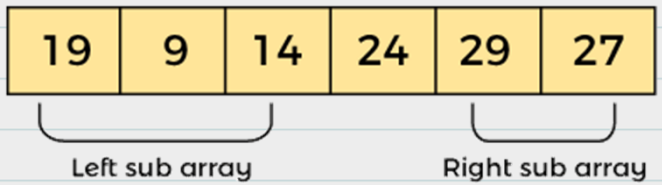


- Now, a[pivot] = 24, a[left] = 14, and a[right] = 24. Pivot is at right, so the algorithm starts from left and move to right.



Cont...

- Now, a[pivot] = 24, a[left] = 24, and a[right] = 24. So, pivot, left and right are pointing the same element. It represents the termination of procedure.
- Element 24, which is the pivot element is placed at its exact position.
- Elements that are right side of element 24 are greater than it, and the elements that are left side of element 24 are smaller than it.



Cont...

- Now, in a similar manner, quick sort algorithm is separately applied to the left and right sub-arrays. After sorting gets done, the array will be -

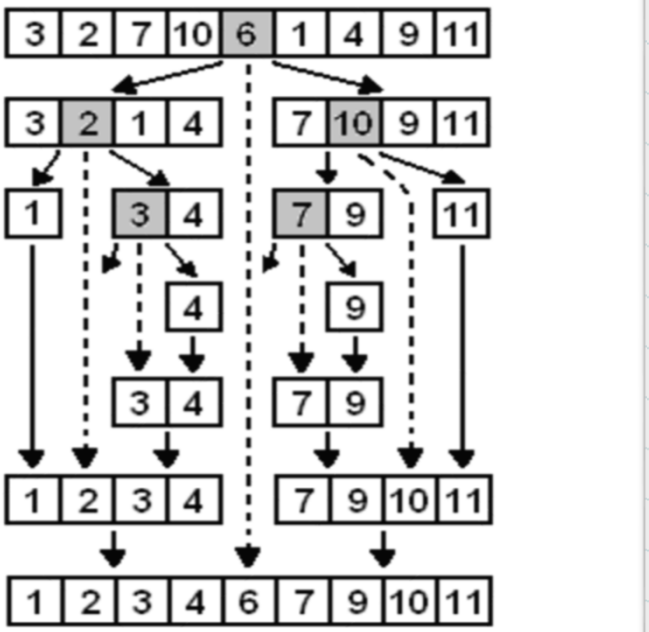
9	14	19	24	27	29

Case	Time Complexity
Best Case	O(n*logn)
Average Case	O(n*logn)
Worst Case	$O(n^2)$

Cont...

Example 2:

3	2	7	10	6	1	4	9	11



5.4 - Merge sort

- **Merge sort** is a sort algorithm that splits the items to be sorted into two groups.
- Recursively sorts each group, and merges them into a final sorted sequence.

It has 3 steps

I. **Divide**: partition the array into two sub list S_1 and S_2 with n/2 element each

II. **Conquer**: then sort sub list S_1&S_2

III. **Combine** :Merge S_1 and S_2 into a unique group.

Cont...

- Example - We take an unsorted array as the following



- The above array of 8 items is divided into two arrays of size 4. This does not change the sequence of appearance of items in the original.



- Now we divide these two arrays into halves



Cont...

- We further divide these arrays and we achieve atomic value which can no more be divided.



- Now, combine them in exactly the same manner as they were broken down. Please note the color codes given to these lists
- We first compare the element for each list and then combine them into another list in a sorted manner.
- We see that **14** and **33** are in sorted positions.

Cont...

- We compare **27** and **10** and in the target list of 2 values we put 10 first, followed by 27.
- We change the order of **19** and **35** whereas **42** and **44** are placed sequentially.



- In the next iteration of the combining phase, we compare lists of two data values, and merge them into a list of found data values placing all in a sorted order.



Cont...

- After the final merging, the list should look like this -



The time complexity of merge sort is

- Best case: $\Theta(n \log n)$
- Worst case: $\Theta(n \log n)$
- Average case: $\Theta(n \log n)$


END
Thank You!